ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC BÁCH KHOA

KHOA ĐIỆN - ĐIỆN TỬ



MÔN HỌC

TRÍ TUỆ NHÂN TẠO TRONG ĐIỀU KHIỂN

"K-MEANS CLUSTERING"

REPORT HOMEWORK 2

Lớp: L01 Nhóm: 05 HK261

GVHD: TS. PHẠM VIỆT CƯỜNG

STT	Sinh viên	MSSV	Điểm
1	Nguyễn Văn Tuấn Anh	2310131	
2	Bùi Tiến Chương	2410411	
3	Nguyễn Tuấn Khanh	2411496	
4	Phạm Đức Quân	2312851	
5	Tạ Hữu Nghĩa	2312278	

TP. Hồ Chí Minh, ngày 23 tháng 09 năm 2026

Mục lục

1 Tóm tắt lý thuyết **2**

2 Example 1 **3**

2.1 Block 1: Tạo tập dữ liệu . . . 3
2.2 Block 2: Khởi tạo và gán nhãn . . . 4
2.3 Block 3: Cập nhật tâm và kiểm tra hội tụ . . . 4
2.4 Block 4: Vòng lặp K-means . . . 5
2.5 Block 5: Kết quả thực nghiệm . . . 5
2.6 So sánh với việc dùng thư viện Scikit-learn . . . 6

3 Example 2 - Image Compression **8**

3.1 Block 1: Nguyên lý biểu diễn màu . . . 8
3.2 Block 2: Chụp ảnh và chuẩn bị dữ liệu . . . 8
3.3 Block 3: Phân cụm màu bằng K-means . . . 10
3.4 Block 4: Tái tạo và lưu ảnh giảm màu . . . 10
3.5 Block 5: Kết quả và nhận xét . . . 11

Danh sách hình vẽ

1 Tập dữ liệu gồm 1500 điểm trước khi phân cụm. . . . 3
2 Kết quả K-means: màu biểu thị nhãn cụm, chấm lớn biểu thị tâm cụm. . . . 6
3 So sánh trực quan kết quả tự cài đặt (trái) và Scikit-learn (phải): màu thể hiện nhãn cụm đã ghép, dấu X là tâm cụm. . . . 7
4 So sánh ảnh webcam trước và sau khi giảm số màu. . . . 11
5 Kích thước tệp JPEG trước và sau khi giảm số màu. . . . 11

1 Tóm tắt lý thuyết

- Hạn chế:

+ Cần biết số lượng cluster, hay là K.

+ Nghiệm cuối phụ thuộc vào việc khởi tạo tâm cụm (centroid) ban đầu.

+ Các cluster cần có số lượng điểm bằng nhau.

+ Các cluster cần có dạng hình tròn.

+ Khi cluster nằm phía trong 1 cluster khác.

2 Example 1

Ví dụ này dùng K-means cho tệp dữ liệu ngẫu nhiên 2 chiều (`k-means-ex1-103.py`). Với $K = 3$, thuật toán lặp lại hai bước: gán mỗi điểm cho tâm gần nhất theo khoảng cách Euclid và cập nhật mỗi tâm bằng trung bình các điểm được gán vào cụm đó. Mục tiêu là giảm tổng bình phương khoảng cách trong các cụm:

$$J = \sum_{i=1}^{1500} \|\mathbf{x}_i - \mu_{z_i}\|_2^2, \qquad z_i \in \{0,1,2\}. \tag{1}$$

Ở đây $\mathbf{x}_i$ là một điểm dữ liệu, z_i là nhãn cụm và μ_{z_i} là tâm của cụm tương ứng.

2.1 Block 1: Tạo tập dữ liệu

Ba nhóm điểm được lấy mẫu từ phân phối chuẩn hai chiều, cùng ma trận hiệp phương sai đơn vị và có trung bình lần lượt là $(2,2)$, $(8,3)$, $(3,6)$. Mỗi nhóm gồm $N = 500$ điểm; sau khi ghép, ma trận X có kích thước $(1500,2)$ và số cụm cần tìm là $K = 3$.

Listing 1: Sinh ba nhóm dữ liệu và chọn số cụm.

```
np.random.seed(18)

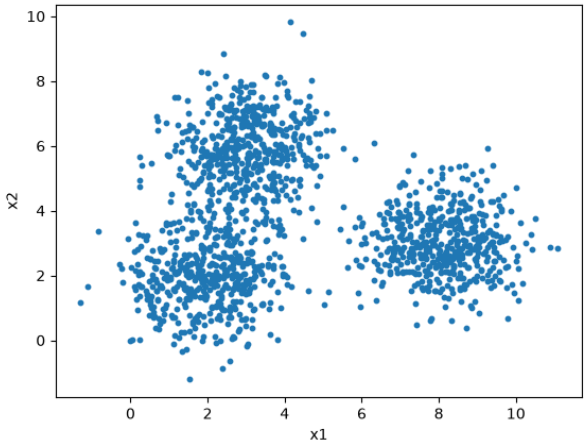
means = [[2, 2], [8, 3], [3, 6]]
cov = [[1, 0], [0, 1]]

N = 500

X0 = np.random.multivariate_normal(means[0], cov, N)
X1 = np.random.multivariate_normal(means[1], cov, N)
X2 = np.random.multivariate_normal(means[2], cov, N)

# X.shape = (N * 3, 2) = (1500, 2)
X = np.concatenate((X0, X1, X2), axis=0)

K = 3
```



Hình 1: *Tập dữ liệu gồm 1500 điểm trước khi phân cụm.*

Hình 1 cho thấy ba vùng điểm tập trung quanh các trung bình đã chọn, đồng thời có một số điểm ở rìa các vùng nằm gần nhau.

2.2 Block 2: Khởi tạo và gán nhãn

Ba hàng khác nhau của X được chọn ngẫu nhiên làm tâm ban đầu . Với mỗi điểm $\mathbf{x}_i$, hàm `kmeans_assign_labels` dùng `cdist` để tính khoảng cách đến từng tâm, rồi lấy chỉ số của tâm gần nhất:

$$z_i = \arg\min_{k\in\{0,1,2\}} \|\mathbf{x}_i - \mu_k\|_2 . \tag{2}$$

Mảng nhãn trả về có 1500 phần tử, mỗi phần tử nhận một giá trị từ 0 đến 2.

Listing 2: Chọn tâm ban đầu và gán điểm cho tâm gần nhất.

```
# randomly pick k rows from X as initial centroids
def kmeans_init_centroids(X, K):
    return X[np.random.choice(X.shape[0], K, replace=False)]

centroids = kmeans_init_centroids(X, K)
print(centroids)
print(centroids.shape) # --> (3, 2)

#-----------------------------------------#
def kmeans_assign_labels(X, centroids):
    # calculate pairwise distances btw data and centroids follow L2 standard
    D = cdist(X, centroids)
    # return index of the closest centroid
    return np.argmin(D, axis=1) # --> array([1, 0, 2, ..., 1, 0, 2]), point i closest to
    centroid j

labels = kmeans_assign_labels(X, centroids)
print(labels.shape) # --> (1500,)
print(labels[:20]) # --> 20 first label
```

2.3 Block 3: Cập nhật tâm và kiểm tra hội tụ

Sau khi gán nhãn, tâm cụm k được thay bằng trung bình của các điểm thuộc cụm đó:

$$\mu_k^{\text{new}} = \frac{1}{|C_k|}\sum_{i\in C_k} \mathbf{x}_i, \qquad C_k = \{i : z_i = k\}. \tag{3}$$

Hàm `has_converged` so sánh tập các tâm cũ và mới; vòng lặp dừng khi chúng bằng nhau. Cách kiểm tra này dùng giá trị dấu phẩy động đúng bằng nhau và không có ngưỡng sai số. Hàm cập nhật cũng giả định mỗi cụm đều có ít nhất một điểm, vì trung bình của cụm rỗng không được xử lý riêng.

Listing 3: Tính tâm mới và xác định điều kiện dừng.

```
def kmeans_update_centroids(X, labels, K):
    centroids = np.zeros((K, X.shape[1]))
    # empty array to store new centroids
    """
    centroids =
    [
        [0, 0],
        [0, 0],
        [0, 0]
```

```
    ]
    """
    for k in range(K):
        Xk = X[labels == k, :]
        centroids[k, :] = np.mean(Xk, axis=0)

    return centroids
#---------------------------------------------#

#---------------------------------------------#
def has_converged(centroids, new_centroids):
    return (
        set([tuple(a) for a in centroids])
        ==
        set([tuple(a) for a in new_centroids])
    )
```

2.4 Block 4: Vòng lặp K-means

Hàm `kmeans` lưu các tâm và nhãn theo từng bước. Mỗi vòng lặp gán lại nhãn, tính tâm mới và kiểm tra hội tụ. Biến `it` chỉ tăng khi tâm mới khác tâm trước đó; vì vậy số được in ra là số lần cập nhật tâm, không tính lượt kiểm tra cuối cùng khi thuật toán dừng.

Listing 4: Vòng lặp gán nhãn, cập nhật tâm và dừng khi hội tụ.

```
def kmeans(X, K):
    centroids = [kmeans_init_centroids(X, K)]
    labels = []
    it = 0
    while True:
        labels.append(kmeans_assign_labels(X, centroids[-1]))
        new_centroids = kmeans_update_centroids(X, labels[-1], K)
        if has_converged(centroids[-1], new_centroids):
            break
        centroids.append(new_centroids)
        it += 1
    return (centroids, labels, it)
```

2.5 Block 5: Kết quả thực nghiệm

Code vẽ các điểm theo nhãn cuối cùng và đánh dấu ba tâm bằng các chấm lớn.

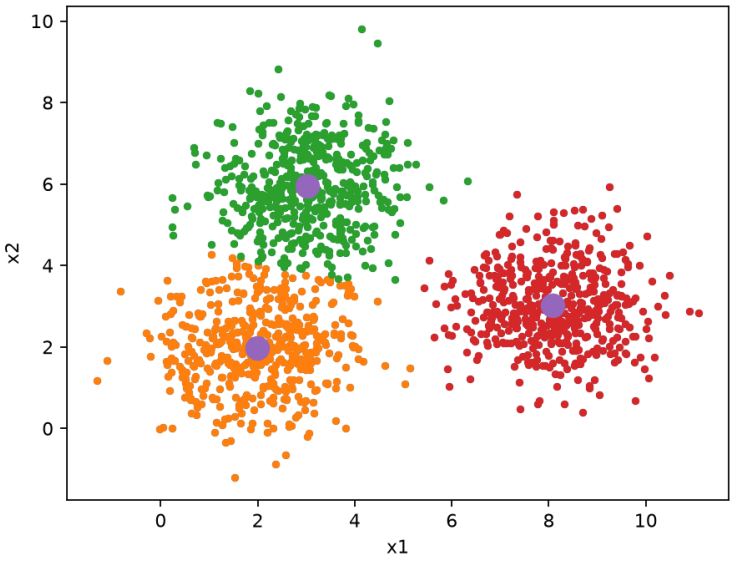
Kết quả chạy là `Number of iterations: 7`; các tâm cuối cùng theo thứ tự nhãn do chương trình trả về là:

$$\begin{aligned} \mu_0 &\approx (1.9835,\ 1.9659), \\ \mu_1 &\approx (3.0270,\ 5.9569), \\ \mu_2 &\approx (8.0748,\ 3.0149). \end{aligned} \tag{4}$$

Các tâm này gần với ba trung bình dùng để sinh dữ liệu. Chỉ số nhãn cụm phụ thuộc vào thứ tự khởi tạo và không mang ý nghĩa thứ bậc.

Listing 5: Hiển thị nhãn cụm, in số lần cập nhật và các tâm cuối.

```python
def kmeans_display(X, labels):
    for k in np.unique(labels):
        plt.scatter(X[labels == k, 0], X[labels == k, 1], s=10)
    final_centroids = centroids[-1]
    plt.scatter(
        final_centroids[:, 0],
        final_centroids[:, 1],
        marker="o",
        s=150
    )
    plt.xlabel("x1")
    plt.ylabel("x2")
    plt.show()
#-----------------------------------------#
(centroids, labels, it) = kmeans(X, K)
print("Number of iterations:", it)
print("Centers found by our algorithm:\n", centroids[-1])
kmeans_display(X, labels[-1])
```



Hình 2: *Kết quả K-means: màu biểu thị nhãn cụm, chấm lớn biểu thị tâm cụm.*

Hình 2 cho thấy ba cụm tương ứng với ba vùng dữ liệu ban đầu. Những điểm nằm ở vùng giao nhau được xếp vào cụm có tâm gần nhất, đúng với quy tắc gán nhãn của thuật toán.

2.6 So sánh với việc dùng thư viện Scikit-learn

Để kiểm tra kết quả, file `k-means-scikit_learn.py` chạy lại chương trình lấy đúng ma trận X như file `k-means-ex1-103.py`, sau đó phân cụm cùng dữ liệu bằng `sklearn.cluster.KMeans` với $K = 3$, `random_state=0`, `n_init=10` và `tol=0`. Trước khi so sánh, chương trình ghép các tâm gần nhau vì số thứ tự nhãn cụm của hai cách chạy có thể khác nhau. Sau khi ghép, ba tâm do Scikit-learn trả về lần lượt là $(1.983497, 1.965881)$, $(3.027029, 5.956861)$ và $(8.074769, 3.014949)$, kết quả gần như trùng với code ban đầu đến sáu chữ số thập phân.

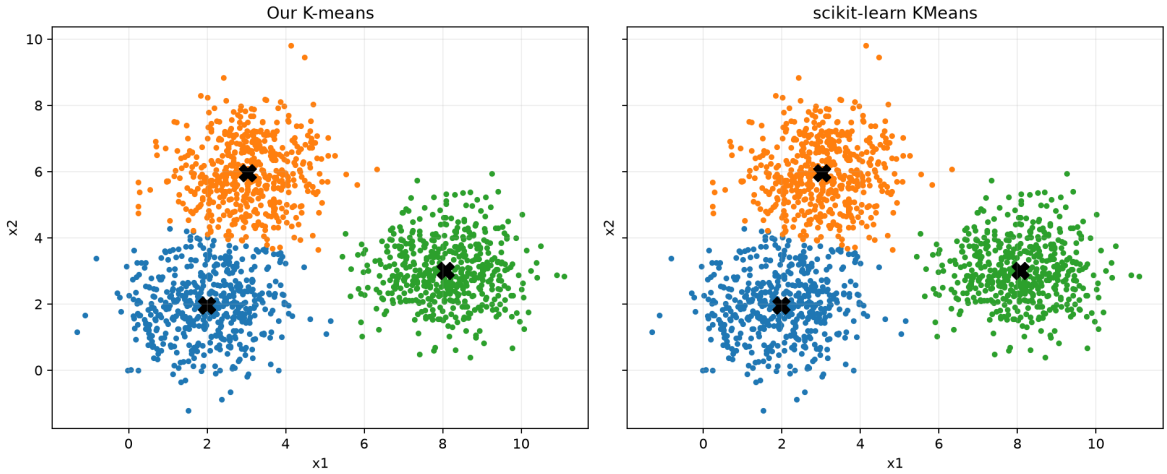
Cả $1500/1500$ nhãn đều khớp sau khi ghép nhãn; chỉ số Adjusted Rand bằng 1.0. Hai giá trị inertia trong Bảng 1 chỉ khác ở sai số làm tròn dấu phẩy động.

Chỉ tiêu	Tự cài đặt	Scikit-learn
Số lần lặp được báo cáo	7	6
Inertia	3005.97947721704	3005.9794772170408

Bảng 1: Kết quả hai cách chạy K-means trên cùng 1500 điểm dữ liệu.

Số lần lặp 7 và 6 không so sánh trực tiếp được: file tự code đếm số lần tâm thay đổi, còn Scikit-learn báo số vòng lặp của lần chạy được chọn trong các lần khởi tạo.

Hai cách cũng khởi tạo tâm khác nhau, file tự code chọn ba điểm ngẫu nhiên từ dữ liệu, còn Scikit-learn chọn ba điểm ngẫu nhiên từ một tập con gồm 10 lần khởi tạo (K-means++). Nhưng đã hội tụ đến cùng kết quả phân cụm trong ví dụ này.



Hình 3: *So sánh trực quan kết quả tự cài đặt (trái) và Scikit-learn (phải): màu thể hiện nhãn cụm đã ghép, dấu X là tâm cụm.*

Hình 3 cho thấy hai cách cho cùng cách chia ba vùng dữ liệu và các tâm nằm ở cùng vị trí. Đây là kiểm tra trực quan phù hợp với kết quả khớp nhãn và inertia ở trên.

3 Example 2 - Image Compression

Ví dụ này dùng K-means để giảm số màu của ảnh chụp từ webcam. Mỗi điểm ảnh được xem là một điểm dữ liệu trong không gian màu RGB. Thuật toán gom các màu gần nhau thành K cụm; khi tái tạo ảnh, màu của mỗi điểm ảnh được thay bằng màu tâm của cụm tương ứng. Mã nguồn nằm trong tệp `k-means-img_compression.py`.

3.1 Block 1: Nguyên lý biểu diễn màu

Trước khi giảm màu: Một điểm ảnh RGB có ba giá trị R, G, B, mỗi giá trị dùng 8 bit, tức $8+8+8=24$ bit/điểm ảnh. Với ảnh 640×480, lượng dữ liệu màu thô là

$$640\times480\times24=7\,372\,800\text{ bit.} \tag{5}$$

Sau khi giảm màu, minh họa với $K=16$: Mỗi điểm ảnh chỉ cần một chỉ số trong tập $\{0,1,2,\ldots,15\}$, nên chỉ số cần 4 bit. Một bảng màu lưu giá trị RGB ứng với từng chỉ số. Chẳng hạn, bảng màu minh họa ban đầu có thể viết gọn như sau:

Chỉ số	Giá trị RGB minh họa
0	[240, 50, 60]
1	[20, 20, 20]
2	[180, 150, 130]
⋮	⋮
15	[90, 100, 120]

Nếu lưu ảnh theo dạng chỉ số màu kèm bảng màu, dung lượng lý thuyết cho $K=16$ là $640\times480\times4+16\times24=1\,229\,184$ bit.

Trong chương trình thực tế, `K = 20`. Cần 5 bit để mã hóa một trong 20 chỉ số màu, nên dung lượng lý thuyết tương ứng là

$$640\times480\times5+20\times24=1\,536\,480\text{ bit.} \tag{6}$$

Các phép tính trên mô tả cách lưu ảnh bằng bảng màu. Chương trình xuất ảnh JPEG, vì vậy dung lượng tệp thực tế còn phụ thuộc cách mã hóa JPEG.

3.2 Block 2: Chụp ảnh và chuẩn bị dữ liệu

Hàm `capture_from_webcam` mở webcam, hiển thị đếm ngược 3 giây, lật khung hình theo chiều ngang rồi lưu ảnh gốc vào `webcam_capture.jpg`. Ảnh do OpenCV đọc có thứ tự kênh BGR; trước khi xử lý và hiển thị, mã chuyển sang RGB. Khối `finally` giải phóng webcam và đóng cửa sổ hiển thị sau khi chụp hoặc gặp lỗi.

Listing 6: Chụp ảnh từ webcam và chuyển thứ tự kênh BGR sang RGB.

```
9  def capture_from_webcam(delay_seconds=3):
10     camera = cv2.VideoCapture(0)
11     if not camera.isOpened():
12         raise RuntimeError("Cant open webcam. Check access!.")
13     try:
14         start_time = cv2.getTickCount()
15         while True:
16             success, frame = camera.read()
17             if not success:
18                 raise RuntimeError("Cant read img from webcam.")
19
```

```
            frame = cv2.flip(frame, 1) # convert to mirror image
            elapsed = (cv2.getTickCount() - start_time) / cv2.getTickFrequency()
            remaining = delay_seconds - int(elapsed)
            if remaining > 0:
                cv2.putText(
                    frame,
                    f"Shoot in {remaining} seconds...",
                    (30, 50),
                    cv2.FONT_HERSHEY_SIMPLEX,
                    1,
                    (0, 255, 0),
                    2,
                )
                cv2.imshow("Webcam - K-means image compression", frame)
                if cv2.waitKey(1) & 0xFF == 27:
                    raise RuntimeError("User cancelled the image capture.")
            else:
                if not cv2.imwrite(image_path, frame):
                    raise RuntimeError(f"Cant save image: {image_path}")
                return cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    finally:
        camera.release()
        cv2.destroyAllWindows()
```

Listing 7: Đặt đường dẫn, chụp ảnh gốc và kiểm tra kích thước ảnh.

```
image_path = os.path.join(
    os.path.dirname(os.path.abspath(__file__)),
    "webcam_capture.jpg",
)

img = capture_from_webcam()
print(img.shape)
# number of possible colors of each pixel: 256^3 = 16,777,216
```

Ảnh được lưu có kích thước 640×480 điểm ảnh. Với ảnh RGB có chiều cao H, chiều rộng W, lệnh `reshape` biến mảng ảnh $(H, W, 3)$ thành ma trận $X \in \mathbb{R}^{HW \times 3}$. Ở đây X có kích thước $(307\,200, 3)$: mỗi hàng là một điểm ảnh, còn ba cột là ba thành phần màu.

Listing 8: Chuyển ảnh thành ma trận các điểm màu RGB và chọn $K = 20$.

```
X = img.reshape(
    img.shape[0] * img.shape[1],
    img.shape[2]
)

"""
1 sample = 1 pixel = 1 row of X = 1 data point = 1 RGB color
features = [R, G, B] = 3 columns of X
X = [data points, features] = [heght * width, 3]
"""
print("X.shape:", X.shape)
```

```
#-------------------------------------------------------------------------------------------#
#-------------------------------------------------------------------------------------------#

K = 20
```

3.3 Block 3: Phân cụm màu bằng K-means

`KMeans(n_clusters=K, random_state=0)` học 20 tâm màu từ các hàng của X. Mỗi tâm là một bộ ba giá trị RGB đại diện cho một nhóm màu. Lệnh `predict(X)` gán cho từng điểm ảnh chỉ số tâm gần nhất; vì X chính là dữ liệu vừa dùng để huấn luyện, mảng nhãn này cũng có thể lấy từ `kmeans.labels_`.

Listing 9: Huấn luyện K-means và lấy nhãn màu cho từng điểm ảnh.

```
K = 20

kmeans = KMeans(
    n_clusters=K,
    random_state=0
    ).fit(X)
# ---> m1, m2, ..., mK = kmeans.cluster_centers_
print(kmeans.cluster_centers_)

print(kmeans.labels_)

labels = kmeans.predict(X) # label for any pixel (old or new --> ok)
# or: labels = kmeans.labels_ # label for data use after fit() (old data only)
print(labels.shape)
```

3.4 Block 4: Tái tạo và lưu ảnh giảm màu

Với mỗi cụm k, chương trình thay màu của các điểm ảnh có nhãn k bằng tâm màu μ_k:

$$\hat{\mathbf{x}}_i = \mu_{z_i}, \qquad z_i \in \{0, 1, \ldots, 19\}. \tag{7}$$

Sau đó ma trận màu được đưa về kích thước ảnh ban đầu và lưu thành `webcam_capture_compressed.jpg`. Trong bộ nhớ, ảnh tái tạo có tối đa 20 màu khác nhau.

Listing 10: Thay màu từng điểm ảnh bằng tâm cụm, khôi phục hình dạng và lưu ảnh.

```
# empty array to store the compressed image
X_compressed = np.zeros_like(X)

for k in range(K):
    X_compressed[labels == k] = kmeans.cluster_centers_[k]

img_compressed = X_compressed.reshape(img.shape)

compressed_image_path = os.path.join(
    os.path.dirname(os.path.abspath(__file__)),
    "webcam_capture_compressed.jpg",
)
plt.imsave(compressed_image_path, img_compressed)
```

3.5 Block 5: Kết quả và nhận xét

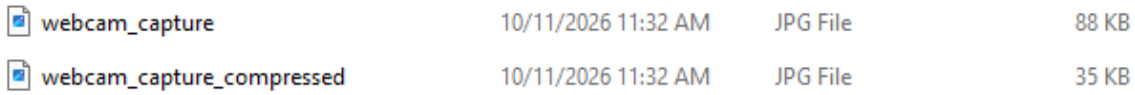


(a) *Ảnh gốc.* **(b)** *Ảnh sau K-means với* $K = 20$.

Hình 4: *So sánh ảnh webcam trước và sau khi giảm số màu.*

Quan sát Hình 4, hình dạng và các vùng màu chính vẫn được giữ lại, nhưng vùng chuyển sắc và một số chi tiết nhỏ trở nên thô hơn vì nhiều màu ban đầu cùng được thay bằng một màu đại diện. Hai tệp JPEG hiện có lần lượt chiếm 85 480 và 34 222 byte; tệp sau xử lý nhỏ hơn khoảng 60%. Đây là dung lượng tệp JPEG đo được, không phải dung lượng lý thuyết của cách lưu chỉ số màu ở Block 1.

Mã còn đếm số màu khác nhau của ảnh gốc và ảnh tái tạo trong bộ nhớ bằng `np.unique`. Ảnh tái tạo trong bộ nhớ có không quá 20 màu, nhưng tệp JPEG khi đọc lại có thể chứa nhiều giá trị RGB hơn do quá trình mã hóa ảnh.



Hình 5: *Kích thước tệp JPEG trước và sau khi giảm số màu.*

Listing 11: Đếm số màu khác nhau trước và sau khi phân cụm trong bộ nhớ.

```
original_colors = np.unique(
    img.reshape(-1, 3),
    axis=0
)

compressed_colors = np.unique(
    img_compressed.reshape(-1, 3),
    axis=0
)
```

Listing 12: Số màu khác nhau của ảnh gốc và ảnh tái tạo trong bộ nhớ.

```
Number of colors in original image: 145484
Number of colors in compressed image: 20
```